```
$ kubectl port-forward service/catalog-service 9001:80
Forwarding from 127.0.0.1:9001 -> 9001
Forwarding from [::1]:9001 -> 9001
```

NOTE The process started by the `kubectl port-forward` command will keep running until you explicitly stop it with Ctrl-C.

Now you can call Catalog Service from your local machine on port 9001, and the request will be forwarded to the `Service` object inside the Kubernetes cluster. Open a new Terminal window, and call the root endpoint exposed by the application to verify that the `polar.greeting` value specified in the ConfigMap is used instead of the default one:

```
$ http :9001/
Welcome to the book catalog from Kubernetes!
```

Try also retrieving the books from the catalog to verify that the PostgreSQL URL specified in the ConfigMap is used correctly:

```
$ http :9001/books
```

When you're done testing the application, stop the port-forward process (Ctrl-C) and delete the Kubernetes objects created so far. Open a Terminal window, navigate to your Catalog Service project (catalog-service), and run the following command, but keep the cluster running, since we're going to use it again soon:

```
$ kubectl delete -f k8s
```

ConfigMaps are convenient for providing configuration data to applications running on Kubernetes. But what if we had to pass sensitive data? In the next section, you'll see how to use Secrets in Kubernetes.